Iterative Deepening Search

Combining BFS's guarantees with DFS's memory efficiency (IDS)

Overview

Iterative Deepening Search (IDS) repeatedly runs a Depth-Limited Search (DFS restricted to a maximum depth) with an increasing depth limit — 0, 1, 2, and so on — until the goal is found. It combines the low memory usage of DFS with the completeness and optimality (for uniform-cost graphs) of BFS, making it a popular choice when the search depth of the solution is unknown.

Algorithm Steps

1. Set depth limit = 0.
2. Run a Depth-Limited DFS from the start node, only exploring nodes up to the limit.
3. If the goal is found within this limit, stop and return the solution.
4. Otherwise, discard the search, increase the depth limit by 1, and restart from the root.
5. Repeat until the goal is found or the limit exceeds the maximum possible depth.

Complexity

Metric	Value
Time Complexity	O(b^d) where b is branching factor, d is depth of solution
Space Complexity	O(b*d) — same as DFS, only stores the current path
Completeness	Complete (finds a solution if one exists, on finite graphs)
Optimality	Optimal when all step costs are equal (like BFS)

Advantages & Disadvantages

Advantages

✓ Uses far less memory than BFS while still finding the shortest path
✓ Does not require knowing the depth of the solution in advance
✓ Combines completeness of BFS with the space efficiency of DFS

Disadvantages

✗ Repeats work from earlier iterations, adding some time overhead
✗ Slightly slower in practice than a single BFS or DFS pass
✗ Less efficient when the branching factor is very small

Typical Use Cases

- Game tree search where the search depth is unknown (e.g., chess engines)
- Puzzle solving with large or unknown search spaces

- Memory-constrained systems that still need optimal solutions
- AI planning problems where solution depth varies

C++ Implementation

C++

```
#include <vector>
using namespace std;

bool dls(int node, int goal, int limit,
         vector<vector<int>>& adj, vector<bool>& visited) {
    if (node == goal) return true;
    if (limit <= 0) return false;

    visited[node] = true;
    for (int neighbor : adj[node]) {
        if (!visited[neighbor] &&
            dls(neighbor, goal, limit - 1, adj, visited)) {
            return true;
        }
    }
    return false;
}

bool ids(int start, int goal, vector<vector<int>>& adj,
         int n, int maxDepth) {
    for (int limit = 0; limit <= maxDepth; limit++) {
        vector<bool> visited(n, false);
        if (dls(start, goal, limit, adj, visited)) {
            return true;  // goal found at this depth
        }
    }
    return false;
}
```